



Práctica 5 – Backtracking

Los objetivos de esta práctica son:

- Convertir el enunciado en lenguaje natural en una descripción de lo pedido en términos de conjuntos de soluciones.
 - Definir soluciones candidatas, válidas y parciales.
 - Para los problemas de optimización, definir una función de valuación para las soluciones válidas.
- Definir una fórmula recursiva para resolver el problema, y demostrar que es correcta con respecto a la especificación.
- Implementar algoritmos de backtracking que implementen la función recursiva. La implementación no necesita ser demostrada; se asume que el pasaje de la fórmula recursiva al algoritmo es directo.
- Modificar el algoritmo para que devuelva el conjunto que logra el óptimo, o una solución válida.
- Entender el árbol de backtracking.
- Implementar podas por factibilidad, optimalidad, etc. y demostrar su correctitud, mostrando que no se pierden soluciones válidas u óptimas.
- Demostrar cotas para la complejidad temporal y espacial de los algoritmos implementados.

Los ejercicios marcados con el símbolo ★ constituyen un subconjunto mínimo de ejercitación. Sin embargo, aconsejamos fuertemente hacer todos los ejercicios.

Los ejercicios marcados con el símbolo ⓘ tienen una ayuda al final de la guía.

Formalización de problemas

Ejercicio 1 (Suma Subconjuntos: formalización) ★

Problema: Suma Subconjuntos

Dado un multiconjunto $C := \{c_1, \dots, c_n\}$ de números naturales y un natural k , queremos determinar si existe un subconjunto de C cuya sumatoria sea k . Vamos a suponer que C está ordenado de alguna forma arbitraria pero conocida (i.e., C está implementado como la secuencia $c_1, \dots, c_n$ o, análogamente, tenemos un iterador de C).

Vamos a traducir el problema [Suma Subconjuntos](#) a otro formalismo. Proponemos las siguientes definiciones:

- a) Las *soluciones (candidatas)* (denotadas como el conjunto **Sols**) se representan con un arreglo de booleanos, es decir, con los vectores $a := (a_1, \dots, a_n)$ de valores binarios; el subconjunto de C representado por a contiene a c_i si y sólo si $a_i = 1$. Escribir el conjunto de soluciones candidatas para $C := \{6, 12, 6\}$ y $k := 12$.
- b) El *predicado de validación* $\text{válida}_{C,k}(a)$ determina si una solución candidata a es *válida* o no. La definimos de la siguiente manera:

$$\text{válida}_{C,k}(a) := \left[\sum_{i=1}^n a_i c_i = k \right].$$

Escribir el conjunto de soluciones candidatas $\text{Sols}_{\text{válidas}}$ para las cuales $\text{válida}_{C,k}(a)$ es verdadero para $C := \{6, 12, 6\}$ y $k := 12$.



- c) Notar que el problema nos pide determinar si el siguiente predicado es verdadero o falso para unos C y k dados:

$$ss(C, k) := [\exists a \in \text{Sols: válida}_{C,k}(a)].$$

Determinar si $ss(C, k)$ es verdadero o falso para $C := \{6, 12, 6\}$ y $k := 12$.

- d) A Cecilia, en cambio, se le ocurrió otro conjunto de soluciones candidatas y de predicado de validación. Sus soluciones candidatas Sols' son todos los posibles conjuntos de números naturales, y dice que una solución candidata sol es válida si cumple lo siguiente:

$$\text{válida}'_{C,k}(\text{sol}) := \left[\text{sol} \subseteq C \wedge \sum_{s \in \text{sol}} s = k \right].$$

Dado un C y un k , ¿qué conjunto es más grande, el conjunto Sols o el conjunto Sols' de Cecilia? Si tuvieran que hacer un algoritmo de fuerza bruta que enumere todas las soluciones candidatas y determine si $ss(C, k)$ es verdadero o falso chequeando cada una, ¿qué conjunto elegirían iterar?

- e) Finalmente, Roberto decide que Sols^* sea exactamente el conjunto $\text{Sols}_{\text{válidas}}$. Su predicado de validación es entonces $\text{válida}^*_{C,k}(a) := \text{True}$. ¿Se les ocurre un algoritmo de fuerza bruta que recorra solamente las soluciones en Sols^* , sin recorrer ninguna otra? (Vale decir que no.)

Ejercicio 2 (MagiCuadrados: formalización) ★

Problema: MagiCuadrados

Un *cuadrado mágico de orden n* es una grilla cuadrada de lado n con los números $\{1, \dots, n^2\}$ tal que todas sus filas, columnas y las dos diagonales suman lo mismo (ver figura). El número que suma cada fila es llamado *número mágico*.

2	7	6
9	5	1
4	3	8

Se pide contar cuántos cuadrados mágicos de orden n existen.

- a) Dar dos propuestas distintas de soluciones candidatas Sols y predicados de validación $\text{válida}_n(a)$ con $a \in \text{Sols}$ para este problema.
- b) Dar un ejemplo de solución candidata no válida y otra válida para cada propuesta.
- c) Para cada propuesta, reescribir al problema en términos de determinar el valor de una función $\text{mc} : \mathbb{N} \rightarrow \mathbb{N}$, donde la entrada es el orden n del cuadrado mágico.

Ejercicio 3 (MaxiSubconjunto: formalización) ★



Problema: MaxiSubconjunto

Dada una matriz simétrica M de $n \times n$ números naturales y un número k , queremos encontrar un subconjunto I de $\{1, \dots, n\}$ con $|I| = k$ que maximice $\sum_{i,j \in I} M_{ij}$. Los índices de las filas y columnas empiezan en 1. Por ejemplo, si $k := 3$ y

$$M := \begin{pmatrix} 0 & 10 & 10 & 1 \\ 10 & 0 & 5 & 2 \\ 10 & 5 & 0 & 1 \\ 1 & 2 & 1 & 0 \end{pmatrix},$$

entonces $I := \{1, 2, 3\}$ es una solución óptima, cuyo valor es $10 + 10 + 10 + 5 + 10 + 5 = 50$.

- Dar una propuesta de conjunto de soluciones candidatas **Sols** y predicado de validación válida $M,k(a)$ con $a \in \text{Sols}$ para este problema.
- En este problema nos interesa encontrar una solución que maximice una fórmula. En otras palabras, estamos buscando encontrar un máximo del conjunto $\text{Sols}_{\text{válidas}}$ según un *orden* $\leq$ entre soluciones válidas. Determinar cuál es ese orden, es decir, determinar cuándo dos soluciones válidas $a, b \in \text{Sols}_{\text{válidas}}$ satisfacen $a \leq b$ y cuándo no.

Repaso de combinatoria

Ejercicio 4 (Quiz combinatorio) ★

- ¿Cuántas hojas tiene un árbol binario completo de n niveles?
a) $\Theta(n)$ b) $\Theta(n^2)$ c) $\Theta(2^n)$ d) $\Theta(n!)$
- Una permutación de un conjunto es una disposición de sus elementos en una secuencia ordenada, sin repetidos. ¿Cuántas permutaciones hay para un conjunto de tamaño n ?
a) $\Theta(n)$ b) $\Theta(n^2)$ c) $\Theta(2^n)$ d) $\Theta(n!)$
- El conjunto de partes de un conjunto A , denotado como $P(A)$, es un nuevo conjunto cuyos elementos son todos los subconjuntos de A . Si A tiene n elementos, ¿cuántos elementos tiene $P(A)$?
a) $\Theta(n)$ b) $\Theta(n^2)$ c) $\Theta(2^n)$ d) $\Theta(n!)$
- Tengo n libros que quiero dividir en k cajas distintas. ¿Cuántas combinaciones tengo?
a) $\Theta(n \cdot k)$ b) $\Theta(n^k)$ c) $\Theta(k^n)$ d) $\Theta\left(\binom{n}{k}\right)$
- Quiero llevar n objetos de mi casa a la facultad, pero en mi mochila solo entran $k < n$. ¿Cuántas combinaciones distintas de objetos podría llevar a la facultad?
a) $\Theta(n \cdot k)$ b) $\Theta(n^k)$ c) $\Theta(k^n)$ d) $\Theta\left(\binom{n}{k}\right)$

Backtracking

Empezamos viendo cómo resolver [Suma Subconjuntos](#) con backtracking.

Ejercicio 5 (Suma Subconjuntos: árbol de backtracking) ★

Definimos una *solución parcial* como un vector $p := (a_1, \dots, a_n)$ de números binarios con $1 \leq i \leq n+1$. El conjunto de todas las soluciones parciales es denotado $\text{Sols}_{\text{parciales}}$. Si $i > 1$, las soluciones *sucesoras* de p son



$0 \oplus p$ y $1 \oplus p$, donde $\oplus$ indica la concatenación. Se le puede dar una *semántica* a estas soluciones parciales: un vector $p := (a_i, \dots, a_n)$ representa un subconjunto del conjunto $C_{\geq i} := \{c_i, \dots, c_n\}$.

- Escribir el conjunto de soluciones parciales para $C := \{6, 12, 6\}$ y $k := 12$, y determinar cuáles son sucesoras de cada una.
- Representar las relaciones de sucesión como un árbol enraizado, donde la solución parcial vacía $()$ es la raíz. A esto se le llama un *árbol de backtracking* del problema.

Ejercicio 6 (Suma Subconjuntos: recurrencia) ★

Sea $\mathcal{C}$ la familia de todos los multiconjuntos de números naturales. Considerar la siguiente función recursiva $\text{ss}_{\text{rec}}: \mathcal{C} \times \mathbb{Z} \rightarrow \text{Bool}$:

$$\text{ss}_{\text{rec}}(\{c_1, \dots, c_n\}, k) := \begin{cases} k = 0 & \text{si } n = 0 \\ \text{ss}_{\text{rec}}(\{c_1, \dots, c_{n-1}\}, k) \vee \text{ss}_{\text{rec}}(\{c_1, \dots, c_{n-1}\}, k - c_n) & \text{si } n > 0 \end{cases}$$

- Demostrar por inducción en n que $\text{ss}_{\text{rec}}(C, k) = \text{ss}(C, k)$ para todos $C \in \mathcal{C}$ y $k \in \mathbb{Z}$. Recordar la definición de $\text{ss}(C, k)$ del ejercicio 1. Para ello, observar que si existe una solución válida para $(\{c_1, \dots, c_n\}, k)$ con $n > 0$, entonces hay dos posibilidades para a_n : o bien $a_n = 0$, o bien $a_n = 1$. En el primer caso, existe un subconjunto de $\{c_1, \dots, c_{n-1}\}$ que suma k ; en el segundo, existe un subconjunto de $\{c_1, \dots, c_{n-1}\}$ que suma $k - c_n$.
- Dibujar el árbol de llamadas recursivas para la entrada $C := \{6, 12, 6\}$ y $k := 12$, indicando claramente la relación entre las distintas componentes del árbol y los conjuntos de soluciones de los ejercicios 1 y 5. Comparar con el árbol de backtracking del ejercicio 5.

Ejercicio 7 (Suma Subconjuntos: implementación) ★

```
1 function subset_sum(C, i, j)
2   if i = 0 then
3     return (j == 0)
4   else
5     return subset_sum(C, i - 1, j) ∨ subset_sum(C, i - 1, j - C[i])
```

- Convencerse que `subset_sum(C, |C|, k)` computa $\text{ss}_{\text{rec}}(C, k)$.
- ¿Cuáles son las complejidades temporal y espacial de `subset_sum(C, |C|, k)`?

Ejercicio 8 (Suma Subconjuntos: poda por factibilidad) ★

Considerar la siguiente *regla de factibilidad* para [Suma Subconjuntos](#): $p := (a_i, \dots, a_n)$ se puede extender a una solución válida sólo si $\sum_{q=i}^n a_q c_q \leq k$.

```
1 function subset_sum_poda(C, i, j)
2   if j < 0 then
3     return false
4   if i = 0 then
5     return (j == 0)
6   else
7     return subset_sum_poda(C, i - 1, j) ∨ subset_sum_poda(C, i - 1, j - C[i])
```

▷ regla de factibilidad



- a) Demostrar que la regla es correcta, es decir, que si una solución parcial p no cumple la regla, entonces todas las soluciones que tengan a p como sufijo no son válidas.
- b) Convencerse de que la implementación incluye la regla de factibilidad.
- c) Dibujar el árbol de llamadas recursivas de `subset_sum_poda(C, |C|, k)` para la entrada $C := \{6, 12, 6\}$ y $k := 12$. Comparar con el árbol de llamadas recursivas de `subset_sum(C, |C|, k)` y con el árbol de backtracking del ejercicio 5. ¿En qué casos se podan ramas del árbol?
- d) ¿Podemos decir algo nuevo sobre las complejidades temporal y espacial en el peor caso de `subset_sum_poda(C, |C|, k)`? ⓘ
- e) Definir otra regla de factibilidad, demostrando que la misma es correcta; no es necesario implementarla.

Ejercicio 9 (Suma Subconjuntos: construcción de solución) ★

Modificar la implementación de `subset_sum` para imprimir el subconjunto de C que suma k , si existe. ⓘ

Ejercicio 10 (Suma Subconjuntos: enumeración) ★

Modificar la implementación del algoritmo del Ejercicio 9 para imprimir **todos** los subconjuntos de C que suman k . ¿Cómo cambia la complejidad temporal?

Ejercicio 11 (MagiCuadrados: backtracking y podas) ★

- a) Para cada una de sus propuestas de soluciones candidatas del Ejercicio 2a), ¿cuántas soluciones habría que generar para encontrar todos los cuadrados mágicos si se utiliza un algoritmo de fuerza bruta?
- b) Definir un conjunto de soluciones parciales para el problema [MagiCuadrados](#) basándose en alguna de sus propuestas de soluciones candidatas en el Ejercicio 2a). Definir una relación de sucesión entre soluciones parciales. ⓘ
- c) Mostrar los primeros dos niveles del árbol de backtracking para $n = 3$.
- d) Demostrar que el árbol de backtracking tiene $\mathcal{O}((n^2)!)$ nodos en peor caso.
- e) Considere la siguiente poda al árbol de backtracking: al momento de elegir el valor de una nueva posición, verificar que la suma parcial de la fila no supere el número mágico. Verificar también que la suma parcial de los valores de las columnas no supere el número mágico. Introducir estas podas al algoritmo e implementarlo en la computadora. ¿Puede mejorar estas podas?
- f) Demostrar que el número mágico de un cuadrado mágico de orden n es siempre $(n^3 + n)/2$. Adaptar la poda del algoritmo del ítem anterior para que tenga en cuenta esta nueva información. Modificar la implementación y comparar los tiempos obtenidos para calcular la cantidad de cuadrados mágicos.

Ejercicio 12 (MaxiSubconjunto: backtracking) ★

- a) Dar una función recursiva para resolver el problema [MaxiSubconjunto](#) utilizando los conjuntos de soluciones y la relación de orden definidos en el Ejercicio 3. Indicar qué es una solución parcial y cómo se extiende cada solución parcial.
- b) Demostrar que la función recursiva es correcta. Dar una *semántica* para la función; es decir, dados unos parámetros fijos, determinar qué significa el resultado de la función, escrito como una fórmula matemática. Demostrar por inducción que la semántica que dieron es correcta para toda entrada.
- c) Calcular la complejidad temporal y espacial del mismo.
- d) Proponer una poda por optimalidad y mostrar que es correcta.
- e) ¿Cómo modificarían su solución si el problema pidiese minimizar $\sum_{i,j \in I} M_{ij}$ en vez de maximizarlo?

Ejercicio 13 (Coloreo) ★

Dado un grafo G y un entero k , el problema de *coloreo* consiste en decidir si es posible pintar cada vértice de G con uno de k colores de manera que no haya dos vértices adyacentes que compartan el mismo color.

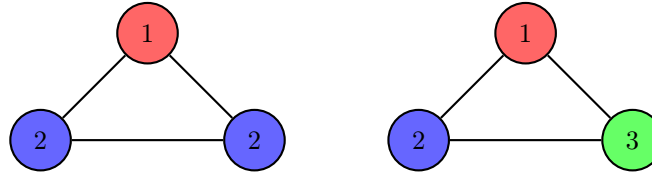


Figura 1: Ejemplo de coloreo de un grafo. El coloreo de la izquierda es inválido, ya que los dos vértices de abajo comparten color y son adyacentes. El coloreo de la derecha sí es válido.

- Dar una función recursiva que resuelva el problema.
- De ser posible, agregar una poda por factibilidad.
- Calcular una cota superior para la complejidad de implementar esa función con backtracking.
- Demostrar que el algoritmo es correcto, definiendo los conjuntos de soluciones, las relaciones entre soluciones, y las semánticas que sean necesarios.
- Modificar el algoritmo para que devuelva un coloreo válido, si es que existe.

Ejercicio 14 (*Viajante de comercio*)

Problema: Viajante de comercio

Sea $G = (V, E)$ un grafo completo (es decir, donde hay aristas entre todo par de nodos) con una función de peso $w : E \rightarrow \mathbb{N}$, que puede interpretarse como el costo de moverse de un nodo a otro. El problema del *viajante de comercio* consiste en encontrar un recorrido que pase por todos los nodos una sola vez de manera de minimizar la suma de los costos, y luego vuelva al primer nodo.

En otras palabras, se busca encontrar una permutación π de $\{1, \dots, n\}$ (el conjunto de nodos) que minimice $w(\pi(n), \pi(1)) + \sum_{i=1}^{n-1} w(\pi(i), \pi(i+1))$. Por ejemplo, si representamos a w como una matriz

$$w = \begin{pmatrix} 0 & 1 & 10 & 10 \\ 10 & 0 & 3 & 15 \\ 21 & 17 & 0 & 2 \\ 3 & 22 & 30 & 0 \end{pmatrix},$$

entonces $\pi(i) = i$ es una solución óptima.

- Diseñar un algoritmo de backtracking para resolver el problema. Demostrar que es correcto indicando claramente cómo se codifica una solución candidata, cuáles soluciones son válidas y cómo están ordenadas, qué es una solución parcial, cómo se extiende cada solución parcial, y la semántica de la función recursiva.
- Calcular la complejidad temporal y espacial del mismo.
- Proponer una poda por optimalidad y mostrar que es correcta.

Ejercicio 15 (*Palabras en cadena*)



Problema: Palabras en cadena

Dada una cadena de letras sin espacios o puntos queremos analizar si se puede subdividir de forma de obtener todas palabras válidas.

Ejemplo:

TDAeslamejormateriadelDC

Se puede subdividir en^c

TDA|es|la|mejor|materia|del|DC

Mientras que

nosvemoc

No tiene una subdivisión válida.

^cAsumimos que TDA y DC están en nuestro diccionario.

Suponiendo que se tiene una función *palabra* que verifica si una cadena c de letras es una palabra en $O(|c|)$.

- Dar una función recursiva que resuelva el problema.
- Calcular una cota superior para la complejidad de implementar esa función con backtracking. **i**
- Demostrar que el algoritmo es correcto, definiendo los conjuntos de soluciones, las relaciones entre soluciones, y las semánticas que sean necesarios.

Ejercicio 16 (Formando parejas)

Dado un grafo $G = (V, E)$, el problema de *apareamiento*, o *matching*, consta en seleccionar un subconjunto $E' \subseteq E$ de aristas tal que ninguna arista $e_1 \in E'$ comparte nodo con otra arista $e_2 \in E'$. Un *apareamiento perfecto* es un apareamiento que cubre todos los vértices del grafo, es decir, para todo $v \in V$ existe alguna arista $e \in E'$ tal que v es uno de los extremos de e .

- Diseñar un algoritmo de backtracking que determine si un grafo G tiene un apareamiento perfecto. Demostrar que es correcto indicando claramente cómo se codifica una solución candidata, cuáles soluciones son válidas, qué es una solución parcial, cómo se extiende cada solución parcial, y la semántica de la función recursiva.
- Calcular la complejidad temporal y espacial del mismo.
- Proponer una poda por factibilidad y mostrar que es correcta.

Ejercicio 17 (ABB óptimo)

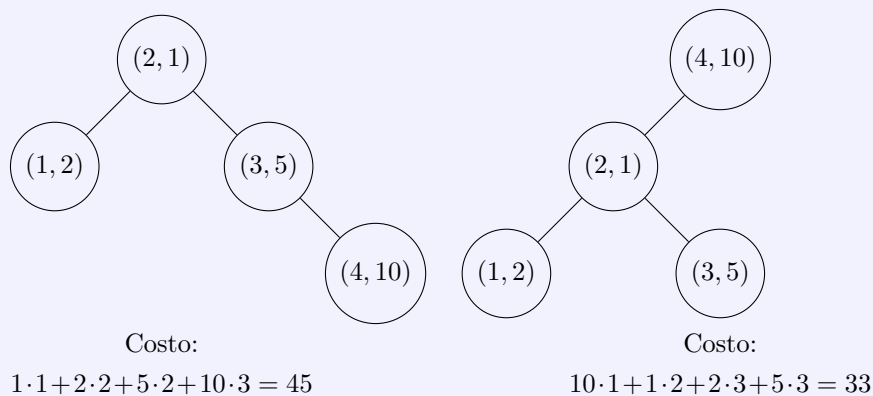


Problema: ABB óptimos

Dado un conjunto de elementos de $[n] = \{1, \dots, n\}$, y una función $f: [n] \rightarrow \mathbb{N}$ que nos da la frecuencia de acceso a dichos elementos, decimos que A es un árbol binario de búsqueda óptimo si este minimiza el costo de todos los accesos dados por f .

En otras palabras el costo de acceder a un elemento i es la cantidad de nodos del árbol que hay que recorrer desde la raíz para llegar a i , incluyendo la raíz e i .

Ejemplo: Si $n = 4$ y $f(1) = 2, f(2) = 1, f(3) = 5, f(4) = 10$ y si notamos a los vértices como $(v, f(v))$ tenemos el siguiente árbol balanceado de búsqueda y un ABB óptimo bajo nuestra métrica:



- Escribir una función recursiva que devuelva el costo de acceder a todos los elementos de un ABB dado usando f .
- Escribir un algoritmo de backtracking que encuentre el ABB óptimo para un f dado.
- Dar una cota superior para la complejidad (Ayuda: pasar de la función recursiva a una recurrencia que solo dependa del tamaño de la entrada).
- Probar que el algoritmo es correcto.

Ejercicio 18 (Cadenas de adición)

Problema: Cadenas de adición

Dado un entero n decimos que $C = \{x_1, \dots, x_k\}$ es una *cadena de adición* de longitud k si cumple lo siguiente:

- $1 = x_1 < x_2 < \dots < x_k = n$; y
- para cada $2 \leq j \leq k$ existen r y s no necesariamente distintos tales que $1 \leq r, s < j$ y $x_r + x_s = x_j$.

Ejemplo: Dado $n = 6$ tenemos las siguientes posibles cadenas

$$1 < 2 < 3 < 5 < 6$$

$$1 < 2 < 3 < 4 < 5 < 6$$

$$1 < 2 < 4 < 6$$

La óptima es la última. Notar que siempre es posible generar una cadena de adición sumando el último número generado $+1$, como se ve en la segunda cadena, es decir que siempre es posible generar al menos una cadena.

- Encontrar un algoritmo de backtracking que encuentre la cadena de adición de longitud mínima para un entero n .



- b) Demostrar su correctitud y complejidad.
- c) Proponer, si no agregaron en el inciso a), al menos una poda por optimalidad y otra por factibilidad.

Ayudas

Calcular cantidad de llamadas a *palabra*.

Ejercicio 15

la posición y se cuentan todos los cuadrados mágicos con esta nueva solución parcial.

- Para establecer el valor de la posición $(i, j + 1)$ (o $(i + 1, 1)$ si $j = n$ e $i \neq n$) se consideran todos los valores que aún no se encuentran en el cuadrado. Para cada valor posible, se establece dicho valor en las primeras j columnas de la fila i .
- La solución parcial tiene los valores de las primeras $i - 1$ filas establecidos, al igual que los valores de

Pueden consultar las transparencias de la teoría sobre árboles de backtracking. También pueden servir las siguientes ideas:

Ejercicio 11

la completitud.

recursiva; tener en cuenta de no suponer que este vector se copia en cada llamada recursiva, porque cambia

Mantener un vector con la solución parcial p al que se le agregan y sacan los elementos en cada llamada

Ejercicio 9

Considerar el caso en el que $\sum_{i=1}^{|C|} c_i \leq k$.

Ejercicio 8